Mechanisms of Action Prediction

A Multi-Model Ensemble for Highly Imbalanced Multi-Label Classification

Marco Merlonghi

Project Work in Machine Learning and Data Mining

Kaggle LISH MoA Prediction Challenge [1]

The Problem

Prediction task

Given the gene-expression and cell-viability profile of human cells exposed to a compound, predict which of 206 mechanisms of action (MoAs) the compound triggers.

- **Multi-label**: a sample can have no positive label, one, or several.
- **Metric**: mean binary log loss over all samples and targets (lower is better).
- **Difficulty**: very sparse positives, strong target imbalance, and *repeated measurements of the same compound*.

Central methodological question

Does the validation score estimate generalisation to compounds that were not seen in training?

Project objectives

- Reproducible cross-validated pipeline; compare neural and tree models; seed bagging.
- Compound-aware validation, and a measure of how optimistic row-wise CV is.
- Tuning that never selects on the scored fold; ensemble re-validated by nested CV.

Data and Label Structure

Training set

- 23,814 rows: 21,948 treated and 1,866 control (7.8%).
- 772 gene-expression (`g-*`) and 100 cell-viability (`c-*`) features.
- `cp_time` (24/48/72 h), `cp_dose` (D1/D2).
- 3,288 distinct compounds among treated rows (median 6 rows, maximum 718), from `train_drug.csv`.

Labels

- Positive rate: **0.373%** of treated-row/target cells.
- 34.2% of treated rows have no label, 57.1% one, 8.7% two or more.
- Positives per target: 1–832 (median 38); 53 targets have fewer than 20.

- Controls have no MoA by definition: excluded from training, predicted as all zeros at inference.
- The test set has **no compound identifier**: overlap with training can only be inferred indirectly.

Runs and Notation

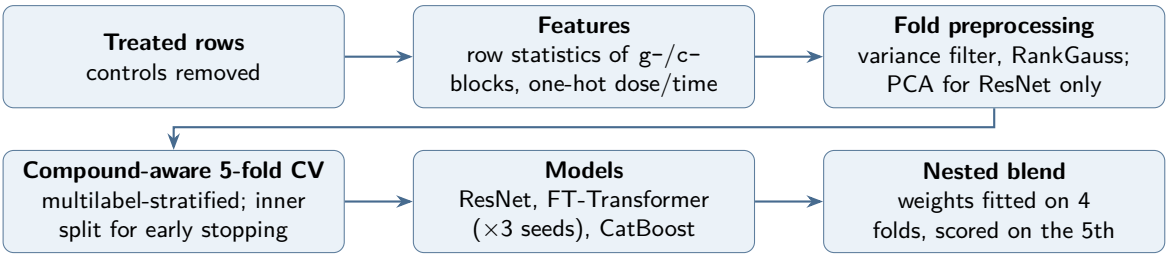
Results refer to three training runs (all with the same code structure, 5 folds, 3-seed bagging):

Run	CV protocol	Hyperparameters	Kaggle	Time (2×T4)
R	row-wise	defaults	yes	≈130 min
H1	hybrid (compound-aware)	tuning v2 (row-wise folds)	no	75 min
H2	hybrid (compound-aware)	tuning v3 (compound-aware folds)	yes	95 min

- H0: an earlier hybrid run (nested blend 0.01720), used only in the fold and compound diagnostics.
- **Deliverables** (Python notebooks): training, inference, three tuning versions, fold diagnostics, offline blend, test-overlap analysis, nearest-neighbour override test.
- Training and inference are separate: the training notebook saves the artifacts, the inference notebook writes `submission.csv` without retraining.

R used an earlier code revision; this matters for the interpretation of the R versus H comparison (slide 13).

End-to-End Pipeline



Inference: fold pipelines and checkpoints are loaded, predictions averaged over folds and seeds, blended with the nested-CV weights; control rows set to 0.

Models and Preprocessing

TabularResNet

- Residual MLP with batch norm and dropout.
- RankGauss → PCA (50 gene and 15 viability components) concatenated to the features.

FT-Transformer [2]

- One token per feature, [CLS] token, self-attention.
- Simplified variant (post-LayerNorm, GELU).
- RankGauss only, no PCA.

CatBoost [3]

- `MultiLogloss`, 250 iterations, one model per fold.
- RankGauss only, no PCA.
- Targets with no training positive: smoothed Jeffreys prior.

Shared training choices

- `BCEWithLogitsLoss` with label smoothing 0.001 [8]; AdamW [6]; `ReduceLROnPlateau`; mixed precision [7].
- Early stopping (patience 8, at most 35 epochs) on an inner 10% split built with the *same grouping rule* as the outer folds; the outer fold is scored once.
- Fold-wise transforms (variance filter, RankGauss, PCA) are fitted on the training fold only [12].

Cross-Validation Protocols

All protocols: 5-fold iterative multilabel stratification [4, 5] on treated rows; the compound id comes from `train_drug.csv`.

Protocol	Unit kept together	Main property
`row`	single row	99.4% of validation rows have their compound in the training fold.
`grouped`	whole compound	No overlap, but compounds have up to 718 rows: fold sizes deviate up to 25.6%.
`hybrid`	compound, except those with $>$ 18 rows (split by row)	9 compounds (9.8% of rows) split by row; fold sizes within 1.4%; 9.8% of validation rows share their compound with training.

Why hybrid?

Pure grouping is unbalanced and pessimistic for targets dominated by a few very large compounds; hybrid trades compound separation for balanced folds. The threshold is a *design choice*, not a proven optimum.

Choosing the Hybrid Threshold

Simulation on the real labels (one splitter seed): compounds with *more* rows than the threshold are split by row.

Scheme	Rows split by row	Max fold-size dev.	Val. rows with seen compound
pure grouped	0%	25.6%	0%
hybrid, $>$ 6 rows	22.0%	3.4%	22.0%
hybrid, $>$ 12 rows	11.9%	3.1%	11.9%
hybrid, $>$ 18 rows	**9.8%**	**1.4%**	**9.8%**
hybrid, $>$ 30, 60 or 100 rows	9.7%	2.8%	9.7%

- The rule: the *largest* threshold that brings fold sizes within a few percent keeps as much of the compound-aware protocol as possible.
- Thresholds 30, 60 and 100 give identical splits (8 compounds); the difference between 1.4% and 2.8% is within what another seed could change.
- The sensitivity of the CV *loss* to the threshold was not measured.

Ensemble and Nested Blending

- Predictions averaged over 3 seeds (and folds) for each network; CatBoost: one model per fold.
- Weights on the simplex via softmax reparameterisation, optimised with Nelder–Mead [11] on out-of-fold predictions.
- **Nested estimate**: weights fitted on four folds, score measured on the fifth, rotating the held-out fold.

$$\hat{p}_{\text{ens}} = w_R \hat{p}_R + w_F \hat{p}_F + w_C \hat{p}_C, \qquad w_R + w_F + w_C = 1$$

Final weights (ResNet / FT / CatBoost)

R: 0.453 / 0.375 / 0.173
H1: 0.371 / 0.405 / 0.225
H2: 0.478 / 0.348 / 0.175

Optimism of the blending step

In-sample minus nested score: $1.6 \cdot 10^{-6}$ (R), $2.8 \cdot 10^{-6}$ (H1), $2.6 \cdot 10^{-6}$ (H2): negligible with three weights.

The nested estimate limits optimism from fitting the weights; it does not remove bias from the CV design or from model selection.

Hyperparameter Tuning: Three Versions

Optuna [9] with TPE sampler [10] and median pruner; the default configuration is trial 0 of every study (v2, v3).

	Folds	**Model selection in a trial**	**Adoption rule**
v1	5 row-wise	best epoch on the *scored* fold (flawed, not used by any reported run)	lowest mean loss
v2	fold 0 search, fold 1 confirm (row-wise)	10% inner split	better than default on both folds, gain $\geq 5 \cdot 10^{-5}$
v3	as v2, compound-aware, own split seed	10% compound-aware inner split	gain $\geq 10^{-4}$ on the confirmation fold *and* wins on more than half of 3 paired seeds

- v3 decides only on the confirmation fold: the search-fold score is the minimum over N trials, hence biased.
- Search: 25/15/15 trials for ResNet/FT/CatBoost, one seed per trial.
- v3 found no FT-Transformer candidate better than the default: it keeps default values.

Selected Hyperparameters

Model	Parameter	Default	H1 (v2)	H2 (v3)
ResNet	Learning rate	10^{-3}	$1.18 \cdot 10^{-3}$	$2.67 \cdot 10^{-3}$
	Dropout	0.2	0.427	0.583
	Weight decay	10^{-5}	$8.5 \cdot 10^{-6}$	$2.66 \cdot 10^{-4}$
	Hidden units / batch	512 / 256	768 / 128	768 / 512
FT-Transformer	LR / dropout / WD	10^{-3} / 0.2 / 10^{-5}	$2.35 \cdot 10^{-3}$ / 0.063 / $9.9 \cdot 10^{-6}$	Default
CatBoost	Depth / L2	5 / 3	4 / 19.0	4 / 56.0
	LR / subsample	0.08 / –	0.1034 / 0.829	0.1034 / 0.829

- In v3, ResNet dropout (0.583), batch size (512) and learning rate ($2.67 \cdot 10^{-3}$) lie at or near the search-space bounds.
  The optimum may be outside the search space.
- CatBoost learning rate and subsample coincide in H1 and H2 (same seeded sampler draw); OOF does not change (0.01782 vs 0.01783).

Results: Cross-Validation

Out-of-fold log loss on treated rows (lower is better); networks use seed-averaged predictions.

Model	**R: row/default**	**H1: hybrid/v2**	**H2: hybrid/v3**
Baseline (per-target prior)	0.02220	0.02233	0.02233
TabularResNet	0.01632	0.01744	0.01735
FT-Transformer	0.01643	0.01736	0.01744
CatBoost	0.01699	0.01782	0.01783
Nested ensemble	$\mathbf{0.01607 \pm 0.00017}$	$\mathbf{0.01714 \pm 0.00031}$	$\mathbf{0.01715 \pm 0.00031}$

- Ensemble improves on the best single model by about 1.5% (R) and 1.2% (H2).
- **Each model contributes** (H2, drop-one nested blend): without ResNet $+1.2 \cdot 10^{-4}$, without FT $+0.8 \cdot 10^{-4}$, without CatBoost $+0.6 \cdot 10^{-4}$; removing any one worsens the score in all five folds.
- Residual correlations are high (0.98 between networks); CatBoost is the most different on positive cells (0.945).

Row-Wise versus Compound-Aware CV

Not a controlled comparison

The +6.7% from R (0.01607) to H2 (0.01715) also reflects code revision, hyperparameters and fold assignment.

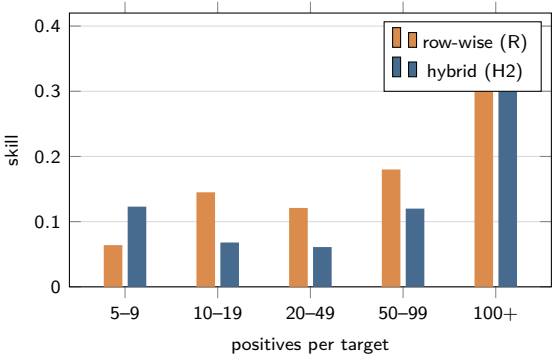
Paired decomposition (blend; run H0 against the row-wise OOF, same rows), bootstrap over compounds:

Rows	Share	Loss row-wise	Loss hybrid	Contribution to gap
Frequent compounds	9.8%	0.00172	0.00198	+0.00003
All other rows	90.2%	0.01762	0.01885	+0.00111

- On ordinary compounds: +0.00123 per row (+7.0%), 95% interval over compounds [+0.00112, +0.00135].
- Almost the whole difference sits on rows that hybrid CV evaluates as *unseen*.
- The worst hybrid fold (fold 4, +4.7%) is *not* explained by frequent compounds: fold spread 0.00091 on all rows, 0.00095 without them.
- Labels are identical across all rows of each of the 8 largest compounds; e.g. 98.9% of `proteasome_inhibitor` positives come from one compound.

Per-Target Analysis (H2 blend)

Skill $= 1 -$ loss/loss of the per-target prior, by number of positives (row-wise vs hybrid):



The per-target baseline differs between protocols: skills are not strictly comparable. Bin 1–4 (2 targets) omitted.

- The 41 targets with ≥ 100 positives carry 47.9% of the loss (skill 0.342).
- The 53 *rare* targets (< 20 positives) carry 6.6% of the loss; skill $+0.080$ [0.056, 0.102]; for 8 of them the model does not beat the prior.
- The blend over-predicts rare targets: expected/observed positives 1.47 below 20 positives.
- 31 targets with ≥ 10 positives come from at most 3 compounds: close to compound-identity labels.

Independent Check: Kaggle Test Set

Two late submissions through the inference notebook (one per configuration).

	R	H2
Nested-CV blend, treated rows	0.01607	0.01715
Rescaled to all test rows ($\times 0.910$)	0.01462	0.01561
Kaggle private	0.01634	0.01643
Kaggle public	0.01865	0.01873
Private above rescaled CV	$+11.8\%$	$+5.3\%$
Public above rescaled CV	$+27.5\%$	$+20.0\%$

- Hybrid CV is closer to the private score and reproduces about 57% of the gap between the row-wise estimate and the private score. This is indirect evidence, not a proof.
- The two submissions differ by $9 \cdot 10^{-5}$ (private) and $8 \cdot 10^{-5}$ (public): **they cannot be ranked**; protocol, tuning and model selection did not move the test score measurably.
- Public and private scores differ by 0.0023 for *both*: the test subsets differ in difficulty.

The test set was never used to tune or select; it did motivate the compound-aware protocols. The rescaling uses the public treated share (0.910): approximate.

Do the Test Compounds Overlap with Training?

Indirect analysis from nearest-neighbour cosine distances (no compound id in the test set).

- **General overlap: not assessable.** Distance separates seen from unseen compounds poorly (AUC 0.516); the mixture estimate must be discarded.
- **Frequent compounds** (5 of 8 analysed are present):

Compound	Test rows assigned	Expected if absent	Test share	Train share
87d714366	133	5.3	3.93%	3.27%
8b87a7a83	23	0.5	0.69%	0.92%
d1b47f29d	24	0.5	0.72%	0.81%
d08af5d4b	36	11.4	0.83%	0.89%
292ab2c28	18	6.2	0.38%	0.88%

- Four are at about their training frequency; 292ab2c28 at about half of it.
- For the other three the assigned rows are close to those expected if absent: presence cannot be established.
- A back-of-the-envelope reweighting with the hybrid losses gives an all-rows estimate of 0.0156–0.0161 against 0.01643 private (residual gap 2–5%): inference, not measurement.

Negative Results

Nearest-neighbour override for frequent compounds

- Pull predictions of rows assigned to a frequent compound towards its label profile (parameters from the training part of each fold).
- Rule precision 0.722, recall 0.809; gain ceiling only $1.7 \cdot 10^{-4}$.
- All four variants **worsen** the OOF loss on all five folds: from $+2.1 \cdot 10^{-4}$ (mixture 0.5) to $+1.47 \cdot 10^{-3}$ (replacement).
- Cost sits on false assignments (663 of 2,381 rows).

Extra networks (run R protocol)

Configuration	Ensemble	Extra weight
+ MLP (PCA)	0.01604	0.015
+ 1D-CNN (PCA)	0.01605	0.082
+ 1D-CNN (raw)	0.01606	0.070
no extra network	0.01605–0.01606	–

Differences $\leq 2 \cdot 10^{-5}$: below the fold standard deviation (0.00017) and comparable to run-to-run noise ($\approx 10^{-5}$).

Takeaway

More components or compound-specific post-processing gave no robust gain on the available validation evidence: the simpler three-model ensemble was kept.

Limitations

1. **Test compound ids unknown:** overlap is inferred only for frequent compounds; ordinary-compound overlap is unassessed, so neither hybrid nor pure grouped CV can be declared the faithful one.
2. **Hybrid threshold is heuristic:** 18 rows chosen from fold balance for one seed; CV-loss sensitivity not measured.
3. **Non-controlled protocol comparison:** R and H runs differ in code, hyperparameters and folds.
4. **Selection bias:** tuning and model choices reused folds overlapping the reported OOF; no untouched hold-out; nested selection of the whole procedure was too costly.
5. **Limited tuning:** one seed per trial, few trials, optimum possibly outside the ResNet search space; no controlled tuned-vs-default comparison under hybrid CV.
6. **Preprocessing:** unsupervised transforms include inner-validation rows; may slightly affect early stopping.
7. **Test evidence:** two submissions, noisy and different public/private subsets; fold standard deviation (0.0003) understates the uncertainty on unseen data.
8. **Reproducibility:** GPU non-determinism ($\approx 10^{-5}$); inference needs the separately saved artifact dataset.

Conclusions

- A reproducible training/inference workflow for a sparse 206-target multi-label task, with separated notebooks and saved artifacts.
- Compound-aware CV exposes a harder generalisation setting than row-wise CV: the extra loss sits almost entirely on ordinary compounds that become "unseen".
- The three-model ensemble beats each component in the nested-CV experiments (each model helps in all five folds).
- Tuning did not measurably improve the hybrid ensemble; the two Kaggle submissions cannot be ranked against each other.
- Nearest-neighbour post-processing and extra networks brought no gain: simplicity was preferred.

Main lesson

For repeated-compound biological measurements, validation design can matter at least as much as model architecture. A CV score must be read together with the assumptions of its split, here the unknown overlap between test and training compounds.

References


Kaggle. *Mechanisms of Action (MoA) Prediction*, 2020. https://www.kaggle.com/competitions/lish-moa


Gorishniy, Y., Rubachev, I., Khrulkov, V., Babenko, A. *Revisiting Deep Learning Models for Tabular Data*. NeurIPS, 2021.


Prokhorenkova, L. et al. *CatBoost: unbiased boosting with categorical features*. NeurIPS, 2018.


Sechidis, K., Tsoumakas, G., Vlahavas, I. *On the Stratification of Multi-Label Data*. ECML PKDD, 2011.


Bradberry, T. *iterative-stratification* (Python package). https://github.com/trent-b/iterative-stratification


Loshchilov, I., Hutter, F. *Decoupled Weight Decay Regularization*. ICLR, 2019.


Micikevicius, P. et al. *Mixed Precision Training*. ICLR, 2018.


Szegedy, C. et al. *Rethinking the Inception Architecture for Computer Vision*. CVPR, 2016 (label smoothing).


Akiba, T. et al. *Optuna: A Next-generation Hyperparameter Optimization Framework*. KDD, 2019.


Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B. *Algorithms for Hyper-Parameter Optimization*. NeurIPS, 2011.


Nelder, J.A., Mead, R. *A simplex method for function minimization*. The Computer Journal 7(4), 1965.


Pedregosa, F. et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, 2011.